



Padrões de Projeto

Singleton, DAO, Factory Method, MVC e Facade

Prof. Ms. Dory Gonzaga Rodrigues

O que são Padrões de Projeto?

Soluções reutilizáveis para problemas comuns no desenvolvimento de software orientado a objetos.

Por que usar?

- Aceleram o desenvolvimento
- Facilitam a manutenção
- Tornam o código mais compreensível
- Promovem boas práticas de engenharia de software

Origem

Popularizados pelo livro

"Design Patterns: Elements of Reusable Object-Oriented Software"

(Gang of Four - GoF).

Classificação

- **Criação** – instanciação de objetos
- **Estrutura** – composição de classes/objetos
- **Comportamento** – comunicação entre objetos

Singleton

Garantir uma única instância

O padrão **Singleton** assegura que uma classe tenha apenas uma instância e fornece um ponto global de acesso a ela.

Quando usar?

- Conexões com banco de dados
- Gerenciador de logs
- Configurações globais
- Pool de threads

No programa exemplo

- A classe DB implementa Singleton para a conexão com o banco de dados.
- O método `getConexao()` cria a conexão apenas na primeira chamada.

```
public class DB {  
    private static Connection conexao;  
  
    public static Connection getConexao() {  
        if (conexao == null) {  
            conexao = DriverManager.getConnection(...);  
        }  
        return conexao;  
    }  
}
```

DAO (Data Access Object)

Separar a lógica de persistência

O padrão **DAO** encapsula o acesso a dados em uma camada separada da aplicação, isolando a lógica de negócio dos detalhes de persistência.

Estrutura

- **Interface DAO** – define operações (insert, update, delete, find)
- **Implementação DAO** – contém o código SQL/JPA
- **Entidade (Model)** – objeto de domínio

No programa exemplo

- AlunoDAO interface
- AlunoDAOImpl implementação com SQL
- Aluno entidade

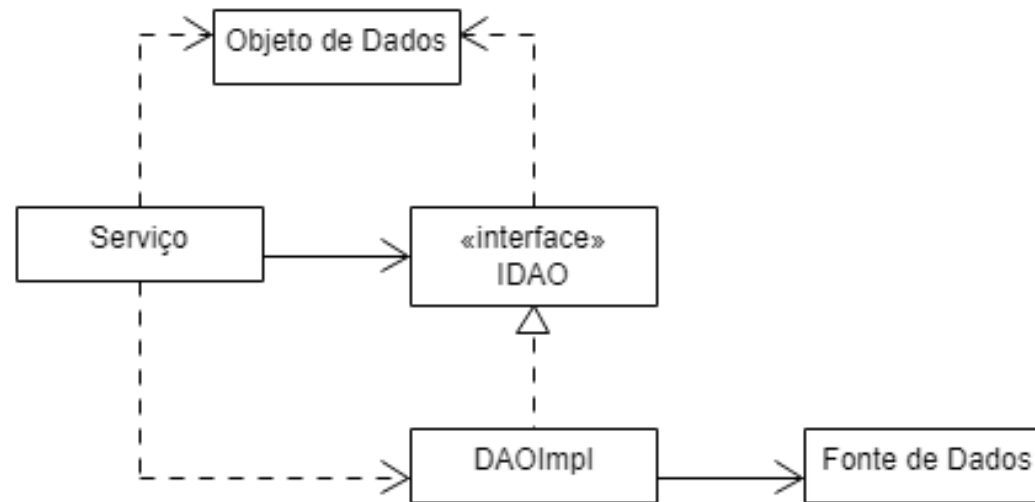
Vantagens do DAO

- Separa a lógica de SQL do restante da aplicação
- Facilita a troca de banco de dados (basta criar uma nova implementação)
- Centraliza o código de persistência, facilitando manutenção e testes

```
public interface CursoDAO {  
    void insert(Curso obj);  
    void update(Curso obj);  
    List<Curso> findAll();  
}
```

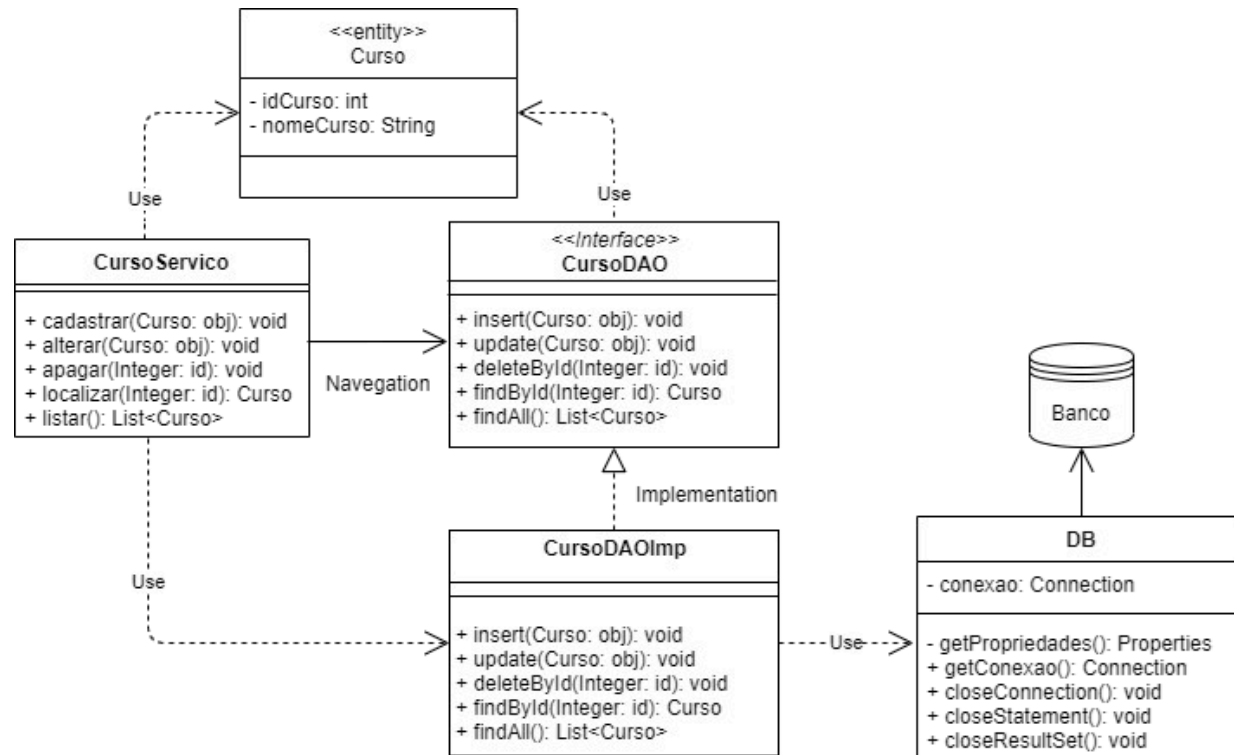
DAO (Data Access Object)

Separar a lógica de persistência



DAO (Data Access Object)

Separar a lógica de persistência



Abstract Factory

Criar famílias de objetos

Abstract Factory fornece uma interface para criar famílias de objetos dependentes ou relacionados sem especificar suas classes concretas.

Vantagens

- Garante consistência entre produtos de uma mesma família
- Facilita a troca de uma família completa de implementações
- Isola as classes concretas do código cliente

No projeto exemplo

- A interface **FactoryDAO** define a criação de todos os DAOs do sistema.
- Cada fábrica concreta implementa os três métodos, produzindo uma família completa de objetos DAO para um banco de dados específico.

```
public interface FactoryDAO {  
    AlunoDAO createAlunoDAO();  
    CursoDAO createCursoDAO();  
    DisciplinaDAO createDisciplinaDAO();  
}
```

Factory Method

Delegar a criação para subclasses

Define uma interface para criar um objeto, mas permite que as subclasses decidam qual classe instanciar.

O Factory Method posterga a instanciação para as subclasses.

Vantagens

- Desacopla o código da classe concreta
- Facilita a extensão com novos tipos de produto
- Aplica o princípio "aberto/fechado" (OCP)

No projeto exemplo

- DAOFactory declara o método fábrica createAlunoDAO().
- Cada fábrica concreta (Postgres, MySQL) implementa esse método, decidindo qual CursoDAO específico instanciar.

```
public class PostgresDAOFactory extends DAOFactory {  
    ...  
    @Override  
    public CursoDAO createCursoDAO() {  
        return new CursoDAOImpl(DB.getConexao());  
    }  
    ...  
}
```


Abstract Factory – Implementação

Fábrica concreta para PostgreSQL

```
public class PostgresDAOFactory implements FactoryDAO {  
  
    @Override  
    public AlunoDAO createAlunoDAO() {  
        return new AlunoPostgresDAO();  
    }  
  
    @Override  
    public CursoDAO createCursoDAO() {  
        return new CursoPostgresDAO();  
    }  
  
    @Override  
    public DisciplinaDAO createDisciplinaDAO() {  
        return new DisciplinaPostgresDAO();  
    }  
}
```

MVC (Model-View-Controller)

Separar responsabilidades em três camadas

O padrão **MVC** divide a aplicação em três componentes interligados:

Model

Gerencia dados e regras de negócio. Notifica observers sobre mudanças.

Exemplos: Aluno, AlunoDAO.

View

Exibe dados ao usuário e captura interações. Não contém lógica de negócio.

Exemplos: TelaAluno, TelaCurso.

Controller

Recebe entradas do usuário, atualiza o Model e escolhe a View apropriada.

Exemplo: AlunoController.

Vantagens do MVC

- Separação clara de responsabilidades
- Facilidade para testar componentes isoladamente
- Permite múltiplas interfaces para o mesmo modelo
- Facilita manutenção e evolução

No projeto exemplo:

As **Views** (telas) chamam métodos dos **Controllers**.

Os **Controlles** por sua vez utilizam os **DAOs** (Model).

O programa principal apenas **instancia a fachada**, que gerencia os fluxos.

MVC – Controller

A camada que orquestra a aplicação

O **Controller** é o intermediário entre a View e o Model.

Ele recebe as requisições do usuário (via View), valida dados, chama as operações do Model e decide qual View será exibida em seguida.

Responsabilidades do Controller

- Receber e interpretar entradas do usuário
- Validar dados antes de enviar ao Model
- Coordenar a atualização do Model
- Selecionar a próxima View

No projeto exemplo

- **CursoController** usa **CursoDAO** para persistir e se comunica com a **TelaCurso** (View).
- O controller não sabe se o banco é PostgreSQL ou MySQL – isso é responsabilidade do DAO.

```
public class CursoController {  
    private CursoDAO cursoDAO;  
  
    public CursoController(FactoryDAO factory) { ... }  
    public void salvar(String nome) { ... }  
    public void atualizar(Integer id, String nome) { ... }  
    public void excluir(Integer id) { ... }  
    public Curso buscarPorId(Integer id) { ... }  
    public List<Curso> listarTodos() { ... }  
}
```

MVC – Model

A camada de dados e regras de negócio

O **Model** representa os dados da aplicação e as regras que garantem a integridade e o comportamento do negócio. Ele é independente da interface com o usuário e pode notificar observers (como Views) sobre mudanças.

Componentes típicos do Model

- **Entidades** (ex: Aluno, Curso e Disciplina)
- **DAOs** – acesso a dados
- **Serviços / regras de negócio**
- Validações que não dependem da interface

No projeto exemplo

- As classes **Aluno**, **Curso**, **Disciplina** e os DAOs correspondentes formam o Model.
- A lógica de negócio deve ficar nas entidades e serviços.
- As validações de entrada podem existir na View ou Controller.

```
public class Curso {  
    private Long id;  
    private String nomeCurso;  
  
    public Curso(String nome) {  
        this.nomeCurso = nome;  
    }  
  
    public boolean isNomeValido() {  
        return nomeCurso != null && nomeCurso.trim().length() >= 3;  
    }  
}
```

MVC – View

A camada de apresentação

A **View** é responsável por exibir dados ao usuário e capturar suas interações (cliques, teclas, etc.). Ela não contém lógica de negócio nem acesso direto a dados – apenas delega ações ao Controller.

Boas práticas para Views

- Não conter SQL ou chamadas a DAOs
- Não implementar regras de validação complexas
- Comunicar-se apenas com o Controller

No projeto exemplo

- **TelaAluno**, **TelaCurso** e **TelaDisciplina** são Views
- Elas apenas coletam dados e chamam os Controllers.

```
public class TelaCurso {  
    private CursoController controller;  
    private Scanner scanner;  
  
    public TelaCurso(CursoController controller,  
                    Scanner scanner) {  
        this.controller = controller;  
        this.scanner = scanner;  
    }  
    ...  
}
```

Facade (Fachada)

Simplificar a interface de um subsistema

O padrão **Facade** fornece uma interface unificada para um conjunto de interfaces dentro de um subsistema. Ele oculta a complexidade interna e reduz o acoplamento entre o cliente e os componentes internos.

Benefícios da Fachada

- Reduz o acoplamento entre o cliente e o subsistema
- Facilita a manutenção e evolução (mudanças internas não afetam o cliente)
- Promove uma camada de serviços bem definida

No projeto exemplo

- A classe SistemaFacade centraliza todos os menus e chamadas aos controllers.
- O método main fica reduzido a uma única linha:
new SistemaFacade().iniciar();
- Toda a complexidade de navegação e criação de objetos fica oculta.

```
public class Program {  
    public static void main(String[] args) {  
        try {  
            SistemaFacade facade = new SistemaFacade();  
            facade.executar();  
        } catch (Exception e) {  
            System.out.println("Erro: " + e.getMessage());  
        } finally {  
            DB.fechaConexao();  
        }  
    }  
}
```

Facade (continuação)

Implementação da SistemaFacade

A classe SistemaFacade concentra a criação dos controllers, telas e o fluxo principal do sistema, atuando como ponto único de acesso.

```
package controller;

import java.util.Scanner;
import model.dao.PostgresDAOFactory;
import view.TelaAluno;
import view.TelaCurso;
import view.TelaDisciplina;

public class SistemaFacade {
    private Scanner scanner;
    private TelaAluno telaAluno;
    private TelaCurso telaCurso;
    private TelaDisciplina telaDisciplina;

    public SistemaFacade() {
        scanner = new Scanner(System.in);
        PostgresDAOFactory factory = new PostgresDAOFactory();
        AlunoController alunoController = new AlunoController(factory);
        CursoController cursoController = new CursoController(factory);
        DisciplinaController disciplinaController = new DisciplinaController(factory);

        telaAluno = new TelaAluno(alunoController, scanner);
        telaCurso = new TelaCurso(cursoController, scanner);
        telaDisciplina = new TelaDisciplina(disciplinaController, scanner);
    }

    public void executar() {
        int opcao;
        do {
```

```
System.out.println("\n===== SISTEMA ESCOLAR =====");
System.out.println("1 - Gerenciar Alunos");
System.out.println("2 - Gerenciar Cursos");
System.out.println("3 - Gerenciar Disciplinas");
System.out.println("0 - Sair");
System.out.print("Opção: ");
opcao = scanner.nextInt();
scanner.nextLine();

switch (opcao) {
    case 1 -> telaAluno.exibirMenu();
    case 2 -> telaCurso.exibirMenu();
    case 3 -> telaDisciplina.exibirMenu();
    case 0 -> System.out.println("Encerrando...");
    default -> System.out.println("Opção inválida!");
}
} while (opcao != 0);
}
```


Observer – Definição e Estrutura

Comunicação desacoplada para eventos específicos

O padrão **Observer** permite que um objeto (sujeito) notifique outros objetos (observadores) sobre mudanças em seu estado.

No projeto exemplo

- Quando um novo curso é cadastrado, o CursoController notifica todos os observadores interessados
- Observador interessado: LogObserver

```
// Interface do Observer
package observer;
import model.entities.Curso;

public interface CursoObserver {
    void onCursoCadastrado(Curso curso);
}
```

```
// Observer concreto
public class LogObserver implements CursoObserver {
    @Override
    public void onCursoCadastrado(Curso curso) {
        System.out.println("[LOG] Curso \"" +
            curso.getNomecurso() + "\" cadastrado em " + dataHora);
    }
}
```

Observer – Sujeito: CursoController

Gerenciando a lista de observadores e notificando

A classe CursoController atua como o **sujeito**.

No projeto exemplo

- A classe **CursoController** mantém uma lista de **CursoObserver**
- Registra novos observadores via **registrarObserver**
- Ao cadastrar um curso (**salvar**), notifica todos os observadores.

```
public class CursoController {  
    private List<CursoObserver> observers = new ArrayList<>  
    ();  
  
    public void registrarObserver(CursoObserver observer) {  
        observers.add(observer);  
    }  
  
    public void salvar(String nome) {  
        Curso c = new Curso(null, nome);  
        cursoDAO.insert(c);  
        for (CursoObserver obs : observers) {  
            obs.onCursoCadastrado(c); // Notificação  
        }  
    }  
}
```

Observer - Registro e Configuração

SistemaFacade como ponto de montagem

A classe SistemaFacade registra os observadores.

No projeto exemplo

- Registra-se um LogObserver.
- Novos observadores podem ser adicionados facilmente, sem alterar o controlador.
Exemplos: notificação por e-mail
- Ao chamar **cursoController.salvar(nome)**, o **LogObserver** será automaticamente notificado e exibirá a mensagem de log com data e hora.

```
public class SistemaFacade {  
    public SistemaFacade() {  
  
        PostgresDAOFactory factory = new PostgresDAOFactory();  
        CursoController cursoController = new CursoController(factory);  
        // Registro do observador de log  
        cursoController.registrarObserver(new LogObserver());  
        // Outros observadores podem ser registrados aqui  
        cursoController.registrarObserver(new EmailObserver());  
    }  
}
```

```
// Exemplo de saída no console
```

```
[LOG] Curso "Engenharia de Software" cadastrado em 21/04/2026 14:35:22
```

Comparação entre os Padrões

Quando usar cada um?

Padrão	Problema que resolve	Exemplo no código
Singleton	Garantir única instância	DB.getConexao()
DAO	Separar persistência da lógica	Curso + CursoDAO + CursoDAOImpl
Factory Method	Encapsular criação de objetos	FactoryDAO + PostgresDAOFactory.createCursoDAO()
MVC	Separar Model, View e Controller	CursoController + TelaCurso
Facade	Simplificar subsistema complexo	SistemaFacade
Observer	Notificar múltiplos objetos sobre mudanças de estado, de forma desacoplada	CursoController + CursoObserver + LogObserver

Resumo

Padrões abordados

Singleton

Única instância. Ex: conexão DB.

DAO

Encapsula persistência.

Factory

Cria objetos de forma centralizada.

MVC

Separação em camadas.

Facade

Interface simplificada para o cliente.

Observer

Notificação desacoplada de eventos.